DAY 1

Module 2

Discovery at Scale: Understanding an Existing Hierarchical Codebase

Before you write a single spec, you have to understand what's already there.

What You'll Learn

1. Explain why reading code alone doesn't scale on a large, hierarchical codebase
2. Use MCP to connect an agent to design documentation and a bug/defect tracker
3. Scope discovery to a service boundary and produce a lightweight dependency map

THE CHALLENGE

Why Code Alone Doesn't Scale

A 50-100 microservice hierarchical estate can't be understood by reading files one at a time.

Undocumented decisions, inconsistent conventions across services, and years of incremental change hide the "why" behind the "what."

Writing a spec against code you don't understand just encodes your misunderstanding into the spec.

THE APPROACH

MCP-Connected Discovery

MCP connects your coding agent to systems outside the codebase: wikis, architecture docs, prior specs, and the bug/defect tracker.

The agent reconstructs a service's architecture and responsibilities from documentation plus code, and can pull in known open defects.

This context becomes the input to Module 3's spec. It doesn't replace the spec itself.

SCOPE IT

Scope discovery to one service boundary, not the whole repository.

Produce a lightweight, one-page service/dependency map as a precursor artifact.

That map, and any defect pulled in, carries forward through the rest of the course.

APPLYING THIS WITH

GitHub Spec Kit

Where this module's concept does, and doesn't, map to a command.

SPEC KIT

No Dedicated Discovery Command

No direct equivalent

Spec Kit's command chain starts at constitution and specify. It has no built-in command for reading unfamiliar code or reconciling it against external documentation before a spec exists.

What to use instead

Discovery happens through your coding agent's own MCP connections (Copilot or Claude Code), one layer below the Spec Kit CLI. The resulting context is what you hand to /speckit.specify as background when you write the spec in Module 3.

APPLYING THIS WITH

OpenSpec

OPENSPEC

Explore: Built for This Moment

`/opsx:explore`

A no-stakes thinking partner: reads your codebase, compares options, and sharpens a fuzzy idea into a concrete plan before any change exists.

With MCP: With MCP connections in place, explore can incorporate design docs and tracker context directly into its investigation.

No artifacts yet: Explore creates no artifacts by itself: it hands off to /opsx:propose once the picture is clear.

Module 2 at a Glance

	GitHub Spec Kit	OpenSpec
Dedicated command	None (handled by the agent's MCP layer)	/opsx:explore
Output	Context fed into the next spec	Investigation only, no artifact
Fit for this course	Discovery happens upstream of the CLI	Purpose-built for brownfield investigation

Key Takeaways

- Discovery precedes specification, always, on a brownfield codebase.
- MCP is the connective tissue between your agent and everything outside the repo.
- OpenSpec's explore command is a natural fit for this step; Spec Kit expects discovery to happen upstream of it.